Data Extraction from Unstructured PDFs

BEGINNER LIBRARIES PANDAS PYTHON PYTHON UNSTRUCTURED DATA

Data Extraction is the process of extracting data from various sources such as CSV files, web, PDF, etc. Although in some files, data can be extracted easily as in CSV, while in files like unstructured PDFs we have to perform additional tasks to extract data from PDF Python.

There are a couple of Python libraries using which you can extract data from PDFs. For example, you can use the **PyPDF2** library for extracting text from PDFs where text is in a sequential or formatted manner i.e. in lines or forms. You can also extract tables in PDFs through the Camelot library. In all these cases data is in structured form i.e. sequential, forms or tables.

However, in the real world, most of the data is not present in any of the forms & there is no order of data. It is present in unstructured form. In this case, it is not feasible to use the above **Python** libraries since they will give ambiguous results. To analyze unstructured data, we need to convert it to a structured form using methods like "python extract data from pdf."

As such, there is no specific technique or procedure for extracting data from unstructured PDFs since data is stored randomly & it depends on what type of data you want to extract from PDF.

Here, I will show you a most successful technique & a python library through which you can extract data from *bounding boxes* in unstructured PDFs and then performing data cleaning operation on extracted data and converting it to a structured form.

In this article you will understand how to extract data from pdf python, extract unstructured data from pdf, extract information from pdf python, and perform unstructured pdf text extraction. Python provides powerful tools to extract data, information and unstructured text from PDF files. Libraries like PyPDF2 and pdfplumber enable extracting structured data as well as parsing unstructured PDF content programmatically.

This article was published as a part of the Data Science Blogathon

Table of contents

- PyMuPDF
- Code
- Data Cleaning and Data Processing
- How to Extract Data from Unstructured PDF Files with Python?
- How to extract information from pdf python?
- Conclusion

PyMuPDF

I have used the **PyMuPDF** library for this purpose. This library provided many applications such as extracting images from PDF, extracting texts from different shapes, making annotations, draw a bounded box around the texts along with the features of libraries like PyPDF2.

Now, I will show you how I extracted data from the bounding boxes in a PDF with several pages.

Here are the PDF and the red bounding boxes from which we need to extract data.

TRAFFIC CRASH REPORT

*DENOTES MANDATORY FIELD FOR SUPPLEMENT REPORT

LOCAL REPORT NUMBER *: 2 1 0 0 9 0 4 8

☒ PHOTOS TAKEN ☒ OH-2 ☐ OH-3
☐ SECONDARY CRASH ☐ OH-1P ☐ OTHER
☐ PRIVATE PROPERTY

LOCAL INFORMATION: 3F

REPORTING AGENCY NAME *: MANSFIELD POLICE
NCIC *: 0 7 0 0 1

HIT/SKIP: 1 - SOLVED, 2 - UNSOLVED
NUMBER OF UNITS: 0 1
UNIT IN ERROR: 98 - ANIMAL, 99 - UNKNOWN: 9 8

COUNTY *: 7 0
LOCALITY *: 1 - CITY, 2 - VILLAGE, 3 - TOWNSHIP: 1
LOCATION: CITY, VILLAGE, TOWNSHIP *: MANSFIELD

CRASH DATE/TIME *: 0 4 2 5 2 0 2 1 0 5 2 8

CRASH SEVERITY: 5
1- FATAL
2- SUSPECTED SERIOUS INJURY
3- MINOR INJURY SUSPECTED
4- INJURY POSSIBLE
5- PROPERTY DAMAGE ONLY

ROUTE TYPE
ROUTE NUMBER
PREFIX: 1 - NORTH, 2 - SOUTH, 3 - EAST, 4 - WEST: 1
LOCATION ROAD NAME: MAIN
ROAD TYPE: S T
LATITUDE: 4 0 . 7 9 0 6 2 0

ROUTE TYPE
ROUTE NUMBER
PREFIX: 1 - NORTH, 2 - SOUTH, 3 - EAST, 4 - WEST
REFERENCE ROAD NAME (ROAD, MILEPOST, HOUSE #): 1150
ROAD TYPE
LONGITUDE: 8 2 . 5 1 3 2 1 0

REFERENCE POINT: 1 - INTERSECTION, 2 - MILE POST, 3 - HOUSE #: 3
DIRECTION FROM REFERENCE: 1 - NORTH, 2 - SOUTH, 3 - EAST, 4 - WEST
DISTANCE FROM REFERENCE
DISTANCE UNIT OF MEASURE: 1 - MILES, 2 - FEET, 3 - YARDS

ROUTE TYPE
IR - INTERSTATE ROUTE (TP)
US - FEDERAL US ROUTE
SR - STATE ROUTE
CR - NUMBERED COUNTY ROUTE
TR - NUMBERED TOWNSHIP ROUTE

ROAD TYPE
AL - ALLEY, AV - AVENUE, BL - BOULEVARD, CR - CIRCLE, CT - COURT, DR - DRIVE, HE - HEIGHTS, HW - HIGHWAY, LA - LANE, MP - MILEPOST, OV - OVAL, PK - PARKWAY, PI - PIKE, PL - PLACE, RD - ROAD, SQ - SQUARE, ST - STREET, TE - TERRACE, TL - TRAIL, WA - WAY

INTERSECTION RELATED
☐ WITHIN INTERSECTION OR ON APPROACH
☐ WITHIN INTERCHANGE AREA
NUMBER OF APPROACHES

ROADWAY
☐ ROADWAY DIVIDED

LOCATION OF FIRST HARMFUL EVENT: 1
1 - ON ROADWAY
2 - ON SHOULDER
3 - IN MEDIAN
4 - ON ROADSIDE
5 - ON GORE
6 - OUTSIDE TRAFFIC WAY
7 - ON RAMP
8 - OFF RAMP
9 - CROSSOVER
10 - DRIVEWAY/ALLEY ACCESS
11 - RAILWAY GRADE CROSSING
12 - SHARED USE PATHS OR TRAILS
13 - BIKE LANE
14 - TOLL BOOTH
99 - OTHER / UNKNOWN

MANNER OF CRASH COLLISION/IMPACT: 1
1 - NOT COLLISION BETWEEN TWO MOTOR VEHICLES IN TRANSPORT
2 - REAR-END
3 - HEAD-ON
4 - REAR-TO-REAR
5 - BACKING
6 - ANGLE
7 - SIDESWIPE, SAME DIRECTION
8 - SIDESWIPE, OPPOSITE DIRECTION
9 - OTHER / UNKNOWN

DIRECTION OF TRAVEL
1 - NORTH
2 - SOUTH
3 - EAST
4 - WEST

MEDIAN TYPE
1 - DIVIDED FLUSH MEDIAN (< 4 FEET)
2 - DIVIDED FLUSH MEDIAN (≥ 4 FEET)
3 - DIVIDED, DEPRESSED MEDIAN
4 - DIVIDED, RAISED MEDIAN (ANY TYPE)
9 - OTHER/UNKNOWN

☐ WORK ZONE RELATED
☐ WORKERS PRESENT
☐ LAW ENFORCEMENT PRESENT
☐ ACTIVE SCHOOL ZONE

WORK ZONE TYPE
1 - LANE CLOSURE
2 - LANE SHIFT/CROSSOVER
3 - WORK ON SHOULDER OR MEDIAN
4 - INTERMITTENT or MOVING WORK
5 - OTHER

LOCATION OF CRASH IN WORK ZONE
1 - BEFORE THE 1st WORK ZONE WARNING SIGN
2 - ADVANCE WARNING AREA
3 - TRANSITION AREA
4 - ACTIVITY AREA
5 - TERMINATION AREA

CONTOUR: 1
1 - STRAIGHT LEVEL
2 - STRAIGHT GRADE
3 - CURVE LEVEL
4 - CURVE GRADE
9 - OTHER/UNKNOWN

CONDITIONS: 1
1 - DRY
2 - WET
3 - SNOW
4 - ICE
5 - SAND, MUD, DIRT, OIL, GRAVEL
6 - WATER (STANDING, MOVING)
7 - SLUSH
9 - OTHER/UNKNOWN

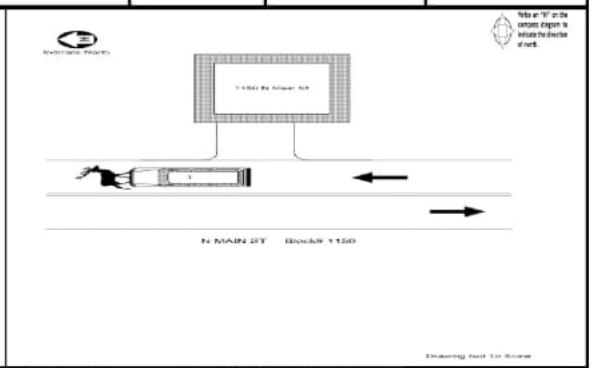
SURFACE: 2
1 - CONCRETE
2 - BLACKTOP, BITUMINOUS, ASPHALT
3 - BRICK/BLOCK
4 - SLAG, GRAVEL, STONE
5 - DIRT
9 - OTHER/UNKNOWN

LIGHT CONDITIONS: 3
1 - DAYLIGHT
2 - DAWN/DUSK
3 - DARK - LIGHTED ROADWAY
4 - DARK - ROADWAY NOT LIGHTED
5 - DARK - UNKNOWN ROADWAY LIGHTING
9 - OTHER / UNKNOWN

WEATHER: 2
1 - CLEAR
2 - CLOUDY
3 - FOG, SMOG, SMOKE
4 - RAIN
5 - SLEET, HAIL
6 - SNOW
7 - SEVERE CROSSWINDS
8 - BLOWING SAND, SOIL, DIRT, SNOW
9 - FREEZING RAIN OR FREEZING DRIZZLE
99 - OTHER/UNKNOWN

NARRATIVE

U#1 was Northbound on N Main St at 1150. A deer crossed the road Eastbound and U#1 struck it causing damage.



CRASH REPORTED DATE/TIME: 0 4 2 5 2 0 2 1 0 5 2 8
DISPATCH DATE/TIME: 0 4 2 5 2 0 2 1 0 5 3 2
ARRIVAL TIME: 0 4 2 5 2 0 2 1 0 5 3 6
SCENE CLEARED DATE / TIME: 0 4 2 5 2 0 2 1 0 5 4 7

REPORT TAKEN BY
☒ POLICE AGENCY
☐ MOTORIST

TOTAL TIME ROADWAY CLOSED
OTHER INVESTIGATION TIME
TOTAL MINUTES: 1 5

OFFICER'S NAME*: CAROLYN YOUNG
OFFICER'S BADGE NUMBER*: 1 7 5 4

CHECKED BY OFFICER'S NAME*: P. WILLIAMS
CHECKED BY OFFICER'S BADGE NUMBER*: 1 0 5 1

☐ SUPPLEMENT (CORRECTION OR ADDITION TO AN EXISTING REPORT SENT TO ODPS)

HSY7001 OH1 1/19 [760-0820]

PAGE 1 OF 4

MOTORIST / NON-MOTORIST

LOCAL REPORT NUMBER *: 2100904 8

UNIT #	NAME: LAST, FIRST, MIDDLE	DATE OF BIRTH	AGE	GENDER
1	STANTO, QUINN NICOLE	04/31/1980	41	F

ADDRESS: CITY, STATE, ZIP	CONTACT PHONE - INCLUDE AREA CODE
1754 BROWNSTONE BLVD - H, TOLEDO, OH 43601	

INJURIES	INJURED TAKEN BY	EMS AGENCY (NAME)	INJURED TAKEN TO: MEDICAL FACILITY (NAME, CITY)	SAFETY EQUIPMENT USED	DOT-COMPLIANT MC HELMET	SEATING POSITION	AIR BAG USAGE	EJECTION	TRAPPED
5				4	☐	1	1	1	1

OL STATE	OPERATOR LICENSE NUMBER	OFFENSE CHARGED	LOCAL CODE	OFFENSE DESCRIPTION	CITATION NUMBER
OH	TK825051				

OL CLASS	ENDORSEMENT SELECT UP TO 2	RESTRICTION SELECT UP TO 3	DRIVER DISTRACTED BY	ALCOHOL/DRUG SUSPECTED	CONDITION	ALCOHOL TEST STATUS	ALCOHOL TEST TYPE	ALCOHOL TEST VALUE	DRUG TEST(S) STATUS	DRUG TEST(S) TYPE	DRUG TEST(S) RESULTS SELECT UP TO 4
4			1	☐ ALCOHOL ☐ MARIJUANA ☐ OTHER DRUG	1	1			1		

UNIT #	NAME: LAST, FIRST, MIDDLE	DATE OF BIRTH	AGE	GENDER

ADDRESS: CITY, STATE, ZIP	CONTACT PHONE - INCLUDE AREA CODE

INJURIES	INJURED TAKEN BY	EMS AGENCY (NAME)	INJURED TAKEN TO: MEDICAL FACILITY (NAME, CITY)	SAFETY EQUIPMENT USED	DOT-COMPLIANT MC HELMET	SEATING POSITION	AIR BAG USAGE	EJECTION	TRAPPED
					☐				

OL STATE	OPERATOR LICENSE NUMBER	OFFENSE CHARGED	LOCAL CODE	OFFENSE DESCRIPTION	CITATION NUMBER

OL CLASS	ENDORSEMENT SELECT UP TO 2	RESTRICTION SELECT UP TO 3	DRIVER DISTRACTED BY	ALCOHOL/DRUG SUSPECTED	CONDITION	ALCOHOL TEST STATUS	ALCOHOL TEST TYPE	ALCOHOL TEST VALUE	DRUG TEST(S) STATUS	DRUG TEST(S) TYPE	DRUG TEST(S) RESULTS SELECT UP TO 4
				☐ ALCOHOL ☐ MARIJUANA ☐ OTHER DRUG							

UNIT #	NAME: LAST, FIRST, MIDDLE	DATE OF BIRTH	AGE	GENDER

ADDRESS: CITY, STATE, ZIP	CONTACT PHONE - INCLUDE AREA CODE

INJURIES	INJURED TAKEN BY	EMS AGENCY (NAME)	INJURED TAKEN TO: MEDICAL FACILITY (NAME, CITY)	SAFETY EQUIPMENT USED	DOT-COMPLIANT MC HELMET	SEATING POSITION	AIR BAG USAGE	EJECTION	TRAPPED
					☐				

OL STATE	OPERATOR LICENSE NUMBER	OFFENSE CHARGED	LOCAL CODE	OFFENSE DESCRIPTION	CITATION NUMBER

OL CLASS	ENDORSEMENT SELECT UP TO 2	RESTRICTION SELECT UP TO 3	DRIVER DISTRACTED BY	ALCOHOL/DRUG SUSPECTED	CONDITION	ALCOHOL TEST STATUS	ALCOHOL TEST TYPE	ALCOHOL TEST VALUE	DRUG TEST(S) STATUS	DRUG TEST(S) TYPE	DRUG TEST(S) RESULTS SELECT UP TO 4
				☐ ALCOHOL ☐ MARIJUANA ☐ OTHER DRUG							

INJURIES

1 - FATAL
2 - SUSPECTED SERIOUS INJURY
3 - SUSPECTED MINOR INJURY
4 - POSSIBLE INJURY
5 - NO APPARENT INJURY

INJURED TAKEN BY

1 - NOT TRANSPORTED / TREATED AT SCENE
2 - EMS
3 - POLICE
9 - OTHER / UNKNOWN

SAFETY EQUIPMENT

1 - NONE USED
2 - SHOULDER BELT ONLY USED
3 - LAP BELT ONLY USED
4 - SHOULDER & LAP BELT USED
5 - CHILD RESTRAINT SYSTEM - FORWARD FACING
6 - CHILD RESTRAINT SYSTEM - REAR FACING
7 - BOOSTER SEAT
8 - HELMET USED
9 - PROTECTIVE PADS USED (ELBOW, KNEES, ETC.)
10 - REFLECTIVE CLOTHING
11 - LIGHTING - PEDESTRIAN / BICYCLE ONLY
99 - OTHER / UNKNOWN

SEATING POSITION

1 - FRONT - LEFT SIDE (MOTORCYCLE DRIVER)
2 - FRONT - MIDDLE
3 - FRONT - RIGHT SIDE
4 - SECOND - LEFT SIDE (MOTORCYCLE PASSENGER)
5 - SECOND - MIDDLE
6 - SECOND - RIGHT SIDE
7 - THIRD - LEFT SIDE (MOTORCYCLE SIDE CAR)
8 - THIRD - MIDDLE
9 - THIRD - RIGHT SIDE
10 - SLEEPER SECTION OF TRUCK CAB
11 - PASSENGER IN OTHER ENCLOSED CARGO AREA (NON-TRAILING UNIT, BUS, PICK-UP WITH CAP)
12 - PASSENGER IN UNENCLOSED CARGO AREA
13 - TRAILING UNIT
14 - RIDING ON VEHICLE EXTERIOR (NON-TRAILING UNIT)
15 - NON-MOTORIST
99 - OTHER / UNKNOWN

AIR BAG

1 - NOT DEPLOYED
2 - DEPLOYED FRONT
3 - DEPLOYED SIDE
4 - DEPLOYED BOTH FRONT / SIDE
5 - NOT APPLICABLE
9 - DEPLOYMENT UNKNOWN

EJECTION

1 - NOT EJECTED
2 - PARTIALLY EJECTED
3 - TOTALLY EJECTED
4 - NOT APPLICABLE

TRAPPED

1 - NOT TRAPPED
2 - EXTRICATED BY MECHANICAL MEANS
3 - FREED BY NON-MECHANICAL MEANS

OL CLASS

1 - CLASS A
2 - CLASS B
3 - CLASS C
4 - REGULAR CLASS (OHIO = D)
5 - M/C MOPED ONLY
6 - NO VALID OL

OL ENDORSEMENT

H - HAZMAT
M - MOTORCYCLE
P - PASSENGER
N - TANKER
Q - MOTOR SCOOTER
R - THREE-WHEEL MOTORCYCLE
S - SCHOOL BUS
T - DOUBLE & TRIPLE TRAILERS
X - TANKER / HAZMAT

GENDER

F - FEMALE
M - MALE
U - OTHER/UNKNOWN

OL RESTRICTION(S)

1 - ALCOHOL INTERLOCK DEVICE
2 - CDL INTRASTATE ONLY
3 - CORRECTIVE LENSES
4 - FARM WAIVER
5 - EXCEPT CLASS A BUS
6 - EXCEPT CLASS A & CLASS B BUS
7 - EXCEPT TRACTOR-TRAILER
8 - INTERMEDIATE LICENSE RESTRICTIONS
9 - LEARNER'S PERMIT RESTRICTIONS
10 - LIMITED TO DAYLIGHT ONLY
11 - LIMITED TO EMPLOYMENT
12 - LIMITED - OTHER
13 - MECHANICAL DEVICES (SPECIAL BRAKES, HAND CONTROLS, OR OTHER ADAPTIVE DEVICES)
14 - MILITARY VEHICLES ONLY
15 - MOTOR VEHICLES WITHOUT AIR BRAKES
16 - OUTSIDE MIRROR
17 - PROSTHETIC AID
18 - OTHER

DRIVER DISTRACTION

1 - NOT DISTRACTED
2 - MANUALLY OPERATING AN ELECTRONIC COMMUNICATION DEVICE (TEXTING, TYPING, DIALING)
3 - TALKING ON HANDS-FREE COMMUNICATION DEVICE
4 - TALKING ON HAND-HELD COMMUNICATION DEVICE
5 - OTHER ACTIVITY WITH AN ELECTRONIC DEVICE
6 - PASSENGER
7 - OTHER DISTRACTION INSIDE THE VEHICLE
8 - OTHER DISTRACTION OUTSIDE THE VEHICLE
9 - OTHER / UNKNOWN

CONDITION

1 - APPARENTLY NORMAL
2 - PHYSICAL IMPAIRMENT
3 - EMOTIONAL (E.G., DEPRESSED, ANGRY, DISTURBED)
4 - ILLNESS
5 - FELL ASLEEP, FAINTED, FATIGUED, ETC.
6 - UNDER THE INFLUENCE OF MEDICATIONS / DRUGS / ALCOHOL
9 - OTHER / UNKNOWN

TEST STATUS

1 - NONE GIVEN
2 - TEST REFUSED
3 - TEST GIVEN, CONTAMINATED SAMPLE / UNUSABLE
4 - TEST GIVEN, RESULTS KNOWN
5 - TEST GIVEN, RESULTS UNKNOWN

ALCOHOL TEST TYPE

1 - NONE
2 - BLOOD
3 - URINE
4 - BREATH
5 - OTHER

DRUG TEST TYPE

1 - NONE
2 - BLOOD
3 - URINE
4 - OTHER

DRUG TEST RESULT(S)

1 - AMPHETAMINES
2 - BARBITURATES
3 - BENZODIAZEPINES
4 - CANNABINOIDS
5 - COCAINE
6 - OPIATES / OPIOIDS
7 - OTHER
8 - NEGATIVE RESULTS

I have tried many python libraries like PyPDF2, **PDFMiner**, **pikepdf**, Camelot, and tabulat. However, none of them worked except **PyMuPDF** to extract data extraction from PDF using Python.

Before going into the code it's important to understand the meaning of 2 important terms which would help in understanding the code.

> Word: Sequence of characters without space. Ex – ash, 23, 2, 3.
>
> Annots: An annotation associates an object such as a note, image, or bounding box with a location on a page of a PDF document, or provides a way to interact with the user using the mouse and keyboard. The objects are called annots. These annotations can be crucial when you extract data from PDF with Python, as they help identify specific data locations.

Please note that in our case the bounding box, annots, and rectangles are the same thing. Therefore, these terms would be used interchangeably.

Also Read: [A Comprehensive Guide to Data Exploration]

First, we will extract text from one of the bounding boxes. Then we will use the same procedure to extract data from all the bounding boxes of pdf.

Code

```
import fitz import pandas as pd doc = fitz.open('Mansfield--70-21009048 - ConvertToExcel.pdf') page1 = doc[0] words = page1.get_text("words")
```

Firstly, we import the fitz module of the PyMuPDF library and pandas library. Then the object of the PDF file is created and stored in doc and the 1st page of the PDF is stored on page1. Using the PyMuPDF library to extract data from PDF with Python, the page.get_text() method extracts all the words from page 1. Each word consists of a tuple with 8 elements.

In words variable, the First 4 elements represent the coordinates of the word, 5th element is the word itself, 6th,7th, 8th elements are block, line, word numbers respectively.

OUTPUT

Extract the coordinates of the first object

```
first_annots=[] rec=page1.first_annot.rect rec #Information of words in first object is stored in mywords mywords = [w for w in words if fitz.Rect(w[:4]) in rec] ann= make_text(mywords) first_annots.append(ann)
```

This function selects the words contained in the box, sort the words and return in form of a string

```
def make_text(words): line_dict = {} words.sort(key=lambda w: w[0]) for w in words: y1 = round(w[3], 1) word = w[4] line = line_dict.get(y1, []) line.append(word) line_dict[y1] = line lines = list(line_dict.items()) lines.sort() return "n".join([" ".join(line[1]) for line in lines])
```

OUTPUT

page.first_annot() gives the first annot i.e. bounding box of the page.

.rect gives coordinates of a rectangle.

Now, we got the coordinates of the rectangle and all the words on the page. We then filter the words which are present in our bounding box and store them in *mywords* variable.

We have got all the words in the rectangle with their coordinates. However, these words are in random order. Since we need the text sequentially and that only makes sense, we used a function make_text() which first sorts the words from left to right and then from top to bottom. It returns the text in string format.

Hurrah! We have extracted data from one annot. Our next task is to extract data from all annots of the PDF which would be done in the same approach.

Extracting each page of the document and all the annots/rectanges

```
for pageno in range(0,len(doc)-1): page = doc[pageno] words = page.get_text("words") for annot in page.annots(): if annot!=None: rec=annot.rect mywords = [w for w in words if fitz.Rect(w[:4]) in rec] ann= make_text(mywords) all_annots.append(ann)
```

all_annots, a list is initialized to store the text of all annots in the pdf.

The function of the outer loop in the above code is to go through each page of PDF, while that of the inner loop is to go through all annots of the page and performing the task of adding texts to all_annots list as discussed earlier.

Printing all_annots provides us the text of all annots of the pdf which you can see below.

OUTPUT

Finally, we have extracted the texts from all the annots/ bounding boxes.

Its time to clean the data and bring it in an understandable form.

Data Cleaning and Data Processing

Splitting to form column name and its values :

```
cont=[] for i in range(0,len(all_annots)): cont.append(all_annots[i].split('n',1))
```

Removing unnecessary symbols *,#,:

```
liss=[] for i in range(0,len(cont)): lis=[] for j in cont[i]: j=j.replace('*','') j=j.replace('#','') j=j.replace(':','') j=j.strip() #print(j) lis.append(j) liss.append(lis)
```

Spliting into keys and values and removing spaces in the values which only contain digits

```
keys=[] values=[] for i in liss: keys.append(i[0]) values.append(i[1]) for i in range(0, len(values)): for j in range(0,len(values[i])): if values[i][j]>='A' and values[i][j]<='Z': break if j==len(values[i])-1: values[i]=values[i].replace(' ','')
```

We split each string based on a new line (n) character to separate the column name from its values. By further cleaning unnecessary symbols like (*, #, □ are removed. Spaces between digits are removed.

With the key-value pairs, we create a dictionary which is shown below:

Converting to dictionary

```
report=dict(zip(keys,values))
```

```
report['VEHICLE IDENTIFICATION']=report['VEHICLE IDENTIFICATION'].replace(' ','')
```

```
dic=[report['LOCALITY'],report['MANNER OF CRASH COLLISION/IMPACT'],report['CRASH SEVERITY']] l=0 val_after=[] for local in dic: li=[] lii=[] k='' extract='' l=0 for i in range(0,len(local)-1): if local[i+1]>='0' and local[i+1]<='9': li.append(local[l:i+1]) l=i+1 li.append(local[l:]) print(li) for i in li: if i[0] in lii: k=i[0] break lii.append(i[0]) for i in li: if i[0]==k:
```

extract=i

```
val_after.append(extract) break report['LOCALITY']=val_after[0] report['MANNER OF CRASH COLLISION/IMPACT']=val_after[1] report['CRASH SEVERITY']=val_after[2]
```

OUTPUT:

Lastly, dictionary is converted to dataframe with the help of pandas.

Converting to DataFrame and exporting to CSV

```
data=pd.DataFrame.from_dict(report)
```

```
data.to_csv('final.csv',index=False)
```

OUTPUT

Now, we can perform analysis on our structured data or export it to excel.

How to Extract Data from Unstructured PDF Files with Python?

To extract data from unstructured PDF files using Python, you can use a combination of libraries such as `PyPDF2` and `NLTK` (Natural Language Toolkit). Here's a general approach:

1. **Install the required libraries by running the following command**

   pip install PyPDF2 nltk

2. **Import the necessary modules in your Python script**

   import PyPDF2
   from nltk.tokenize import word_tokenize

```
from nltk.corpus import stopwords
```

3. **Load the PDF file using PyPDF2**

```
def extract_text_from_pdf(pdf_path):
pdf_file = open(pdf_path, 'rb')
pdf_reader = PyPDF2.PdfFileReader(pdf_file)
num_pages = pdf_reader.numPages

text = ""
for page_num in range(num_pages):
page = pdf_reader.getPage(page_num)
text += page.extract_text()

pdf_file.close()
return text
```

4. **Preprocess the extracted text by removing stopwords and non-alphanumeric characters**

```
def preprocess_text(text):
tokens = word_tokenize(text.lower())
stop_words = set(stopwords.words('english'))
cleaned_tokens = [token for token in tokens if token.isalnum() and token not in stop_words]
cleaned_text = ' '.join(cleaned_tokens)
return cleaned_text
```

5. **Call the functions and extract the data from the PDF**

```
pdf_path = "path/to/your/pdf/file.pdf"
extracted_text = extract_text_from_pdf(pdf_path)
preprocessed_text = preprocess_text(extracted_text)

# Process the preprocessed text further as per your specific requirements
# such as information extraction, entity recognition, etc.
```

This is a basic example to get you started. Depending on the structure and content of your PDF files, you may need to apply additional techniques for more accurate and specific data extraction.

How to extract information from pdf python?

To extract information from PDF files using Python, several libraries can be utilized, each with its own strengths and methods. Below are some popular libraries and examples of how to use them effectively.

Libraries for PDF Extraction

1. **PyPDF2**: A widely used library for reading and manipulating PDF files. It allows you to extract text and metadata from PDFs.
2. **PDFMiner**: This library excels in extracting structured data from PDFs, making it suitable for complex layouts.
3. **PyMuPDF (fitz)**: A powerful library that supports various file formats and provides efficient text extraction and manipulation capabilities.

4. **PDFQuery**: This library uses CSS-like selectors to extract data from PDFs, making it user-friendly for specific data retrieval.
5. **IronPDF**: A commercial library that simplifies PDF data extraction and manipulation, including interactive forms.

Example Code Snippets

Using PyPDF2

```
from PyPDF2 import PdfReader # Create a PDF reader object reader = PdfReader('example.pdf') # Print the number of pages print(len(reader.pages)) # Extract text from the first page text = reader.pages[0].extract_text() print(text)
```

Conclusion

Data extraction from PDFs, especially unstructured ones, can be challenging, but Python offers powerful libraries to simplify this process. While libraries like PyPDF2 and Camelot work well for structured data, PyMuPDF excels in handling unstructured PDFs by extracting data extraction from bounding boxes. This approach, combined with effective data cleaning and processing techniques, enables the conversion of unstructured data into a structured form suitable for analysis. By leveraging these tools, you can efficiently extract, clean, and utilize data from a variety of PDF formats, enhancing your data science projects.

Hope you like the article on how to extract data from PDF using Python. These libraries allow you to extract unstructured data from PDF, making unstructured PDF text extraction a breeze and helping you extract information from PDF with ease.

The media shown in this article are not owned by Analytics Vidhya and are used at the Author's discretion.

Frequently Asked Questions

Q1. How do I extract specific data from a PDF?

A. To extract specific data from a PDF, you can use Python libraries like PyMuPDF or PyPDF2. These libraries allow you to search for and extract text or other elements from specific locations within the PDF.

Q2. How do I extract form data from a PDF?

A. You can extract form data from a PDF using the PyPDF2 or pdfplumber libraries in Python. These libraries can read and extract values from form fields in a PDF.

Q3. Can I extract data from PDF to Excel?

A. Yes, you can extract data from a PDF to Excel using libraries like PyMuPDF, Camelot, or tabula-py to read the PDF and extract the data, then use the pandas library to organize and export the data to an Excel file.

Q4. How to automatically extract data from PDF?

A. To automatically extract data from a PDF, you can use a combination of Python libraries such as PyPDF2 for text extraction and pandas for data processing. You can write a script that reads the PDF, processes the data, and saves it in the desired format, such as CSV or Excel. This script can be scheduled to run automatically using tools like cron jobs or task schedulers.

Article Url - https://www.analyticsvidhya.com/blog/2021/06/data-extraction-from-unstructured-pdfs/

Ashish